SQL Practice Assignment 4: Aggregation & Grouping

Topic: Statistical Functions (COUNT, SUM, AVG, MIN, MAX), GROUP BY, and HAVING

Role: Junior Data Analyst at "TechTrendz Retail"

Part 1: Conceptual Overview

Context: The executive team wants a "State of the Business" report. They don't care about individual items right now; they want big-picture numbers. They want to know averages, totals, and performance per category.

Key Concept - The "HAVING" Clause:

- Use WHERE to filter **raw rows** (e.g., "Show me red shoes").
- Use HAVING to filter **aggregated groups** (e.g., "Show me categories where the average price is > $50").

Part 2: Statistical Analysis Questions

Write SQL queries to solve the following business requests.

A. Global Statistics (Single Row Results)

These queries will return just one row representing the entire dataset.

1. **Total Product Count:** How many unique products do we carry in total? ○ Use the COUNT() function.
   USE techtrendz_retail;
   SELECT COUNT(*) AS total_products
   FROM products;

2. **Average Market Price:** What is the average price of all items in our inventory? ○ Use the AVG() function. Alias the result as avg_market_price.

   SELECT AVG(price) AS avg_market_price
   FROM products;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

avg_market_price
261.262941

3. **Price Range:** What are the cheapest and most expensive prices we offer? ○ Select both MIN(price) and MAX(price) in the same query.

```
SELECT MIN(price) AS cheapest_price,
       MAX(price) AS highest_price
FROM products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cheapest_price	highest_price
15.00	1500.00

4. **Total Stock Volume:** What is the total number of individual units we have in our warehouse across all products?
   ○ Use the SUM() function on the stock_quantity column.

```
SELECT SUM(stock_quantity) AS total_stock_units

FROM products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_stock_units
1135

B. Grouping Data (The "GROUP BY" Clause)

Management often asks for data "Per Category" or "Per Type". This is your cue to use GROUP BY.

5. **Stock per Category:** We need to know which categories are taking up the most space.
   - Select category and the sum of stock_quantity.
   - Group the results by category.

```
SELECT category,
    SUM(stock_quantity) AS total_stock
FROM products
GROUP BY category;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	total_stock
Electronics	255
Furniture	20
Accessories	160
Apparel	330
Fitness	200
Appliances	40
Kitchen	80
NULL	50

6. **Average Rating per Category:** Which product types do customers love the most?
   - Select category and the average rating.
   - Group by category.
   - *Bonus:* Round the average to 2 decimal places if your SQL dialect supports it (optional).

```
SELECT category,
    ROUND(AVG(rating),2) AS avg_rating
FROM products
GROUP BY category;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	avg_rating
Electronics	4.55
Furniture	4.25
Accessories	4.15
Apparel	4.25
Fitness	4.30
Appliances	4.40
Kitchen	4.20
NULL	3.50

C. Filtering Groups (The "HAVING" Clause)

Now we filter the results of your groups. Remember: You cannot use WHERE on an aggregate calculation like SUM or AVG.

7. **High-Volume Categories:** We only want to focus on major categories.
   - Select category and total stock.
   - Group by category.
   - Filter to show **only** categories where the total stock is **greater than 100**.

```
SELECT category,
    SUM(stock_quantity) AS total_stock
FROM products
GROUP BY category
HAVING SUM(stock_quantity) > 100;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	total_stock
Electronics	255
Accessories	160
Apparel	330
Fitness	200

8. **Premium Categories:** Which categories have expensive items on average?
   - Select category and average price.
   - Group by category.
   - Filter to show **only** categories where the AVG(price) is **greater than $50**.

```
SELECT category,
    AVG(price) AS avg_price
FROM products
GROUP BY category
HAVING AVG(price) > 50;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	avg_price
Electronics	563.711429
Furniture	92.500000
Apparel	52.495000
Appliances	75.500000

D. Functions in Other Clauses (Sorting by Statistics)

9. **Category Popularity Ranking:**
   - We want a list of categories sorted by how many **types** of products they contain.
   - Select category and the count of products (count of rows).
   - Group by category.
   - Order the results by the count in **Descending** order (highest count first).

```
SELECT category,
    COUNT(*) AS product_count
FROM products
GROUP BY category
ORDER BY product_count DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	product_count
Electronics	7
Furniture	2
Accessories	2
Apparel	2
Fitness	1
Appliances	1
Kitchen	1
NULL	1